

Организационные вопросы

Оценка лабораторных работ

Лабораторная работа оценивается по 2-балльной шкале (зачтено/не зачтено). Для получения положительной оценки достаточно выполнить работу с соблюдением минимальных требований. Выполнение дополнительных заданий не отменяет необходимость соблюдения минимальных требований.

Минимальные требования

1. Код должен компилироваться без предупреждений при максимальном уровне предупреждений. Для компилятора GCC это набор флагов: `-Wall -Wconversion -Wextra -Wpedantic`. Для компилятора MSVC (Visual Studio) это флаг `/W4`. Для других компиляторов согласуйте настройки компиляции с преподавателем.

2. Нельзя использовать глобальные переменные (константы допустимы).

3. В коде должен использоваться только полноценный английский язык (транслит запрещен). Русский язык разрешено использовать только в комментариях.

4. Запрещается комментировать каждую строчку кода. Допустим один краткий комментарий на блок кода. Разрешается комментировать одиночную строчку кода, только если она действительно делает что-то неожиданное и хитрое (но помните, хитрый код — плохой код!).

5. Весь код должен удовлетворять единому стилю программирования. Сам стиль можно выбирать по своему вкусу (см., например, [Google C++ Style Guide](#), [GNU C Style](#) и другие). То есть запрещено, например, называть одну функцию в стиле [CamelCase](#), а другую — в стиле [snake_case](#). Исключение допустимо только для названия функции `main`, которое всегда пишется в нижнем регистре. Данные требования предъявляются к любым именуемым сущностям в программе — к функциям, методам, классам, локальным переменным, параметрам функций и методов, названиям файлов и так далее.

Лабораторная работа № 3: Графы

В рамках лабораторной работы **разрешено** использование STL-контейнеров, а также классов и функций из заголовочных файлов <memory>, <iterator>, <functional>, <algorithm> и <numeric>.

Запрещено использование функций ручного управления памятью (malloc, free, new и delete).

Работу с памятью делегировать STL-контейнерам и «умным указателям» std::unique_ptr/std::shared_ptr.

Выполнение лабораторной работы состоит из 2 частей: написания класса и демонстрационной программы в соответствии и решения практической задачи согласно варианту с помощью написанного вами класса (распределение вариантов приведено в приложении 1).

Задание

Реализовать шаблонный класс, моделирующий **ориентированный** граф со следующим *примерным* интерфейсом (можно найти более эффективные решения):

```
template<typename Vertex, typename Distance = double>
class Graph{
public:
    struct Edge{/*...*/}

    //проверка-добавление-удаление вершин
    bool has_vertex(const Vertex& v) const;
    bool add_vertex(const Vertex& v); // false если вершина
уже есть
    bool remove_vertex(const Vertex& v);
    std::vector<Vertex> vertices() const;

    //проверка-добавление-удаление ребер
    void add_edge(const Vertex& from, const Vertex& to,
                  const Distance&
d);
    bool remove_edge(const Vertex& from, const Vertex& to);
    bool remove_edge(const Edge& e); //с учетом расстояния
    bool has_edge(const Vertex& from, const Vertex& to) const;
    bool has_edge (const Edge& e) const; //с учетом расстояния
в Edge

    //получение всех ребер, выходящих из вершины
    std::vector<Edge> edges(const Vertex& vertex);

    size_t order() const; //порядок
    size_t degree(const Vertex& v) const; //степень вершины
    bool is_connected() const; //является ли граф сильно
СВЯЗНЫМ
```

```

//поиск кратчайшего пути
std::vector<Edge> shortest_path(const Vertex& from,
                               const Vertex& to)
const;
//обход
std::vector<Vertex> walk(const Vertex& start_vertex,
std::function<void(const Vertex&)> action) const;

```

Вторым параметром должна передаваться функция, которая будет выполняться для каждой вершины, встреченной при обходе. С помощью такой функции а) распечатайте результат обхода; б) получите вектор вершин, встреченных при обходе.

Граф может быть содержать петли и иметь несколько ребер между двумя вершинами.

Способ хранения, методы обхода графа и поиска кратчайшего выбирается согласно варианту (приложение 1).

Решение задачи должно происходить вне класса.

Напишите программу, демонстрирующую возможности вашего класса.

Задачи:

1. Пусть дан связный граф, в котором узлы – это торговые точки. Необходимо превратить одну из торговых точек в склад. Цена доставки от склада в точку зависит от расстояния. Найдите оптимальную с точки зрения максимальных затрат точку (т.е. точку, для которой максимальное расстояние до любой другой точки минимально).

2. Пусть дан некоторый связный граф, в котором узлы – это травмпункты. Предположим, в преддверии зимы вы хотите улучшить покрытие города. Для этого вам необходимо понять, какой травмпункт находится дальше всего от своих прямых соседей (т.е. средняя длина связанных с ним ребер максимальна). Напишите функцию, которая находит такой травмпункт.

3. Пусть дан связный граф, в котором узлы – это торговые точки. Необходимо превратить одну из торговых точек в склад. Очевидно, склад должен иметь минимальное среднее расстояние до остальных точек. Напишите функцию, которая находит такую точку.

Примечание

В случае возникновения сомнений касемо зачета по лабораторной работе преподаватель вправе попросить решить дополнительную задачу в аудитории на основе написанного класса. Задача выдаётся не из вышеуказанного списка.

Приложение 1. Распределение вариантов

Class - 2 lab - использовать готовый класс из второй лабораторной работы

Номер в списке группы	Способ представления графа	Контейнер для хранения	Обход	Поиск кратчайшего расстояния	Задача
1	Class - 2 lab	-	В ширину	Дейкстры	1
2	Матрица смежности	vector	В глубину	Беллмана-Форда	2
3	Списки смежности	vector	В ширину	Беллмана-Форда	3
4	Class - 2 lab	-	В глубину	Дейкстры	3
5	Списки смежности	list	В ширину	Дейкстры	2
6	Матрица смежности	list	В глубину	Беллмана-Форда	1
7	Матрица смежности	vector	В ширину	Беллмана-Форда	1
8	Матрица смежности	vector	В глубину	Дейкстры	2
9	Списки смежности	list	В ширину	Дейкстры	3
10	Списки смежности	vector	В глубину	Беллмана-Форда	3
11	Class - 2 lab	-	В ширину	Беллмана-Форда	2
12	Списки смежности	vector	В глубину	Дейкстры	1
13	Матрица смежности	list	В ширину	Дейкстры	1
14	Матрица смежности	vector	В глубину	Беллмана-Форда	2
15	Списки смежности	vector	В ширину	Беллмана-Форда	3
16	Матрица смежности	list	В глубину	Дейкстры	3
17	Списки смежности	vector	В ширину	Дейкстры	2
18	Class - 2 lab	-	В глубину	Беллмана-Форда	1
19	Списки смежности	vector	В ширину	Беллмана-Форда	1
20	Матрица смежности	vector	В глубину	Дейкстры	2
21	Списки смежности	list	В ширину	Дейкстры	3
22	Class - 2 lab	-	В глубину	Беллмана-Форда	3
23	Class - 2 lab	-	В ширину	Беллмана-Форда	2
24	Списки смежности	vector	В глубину	Дейкстры	1
25	Матрица смежности	list	В ширину	Дейкстры	2
26	Списки смежности	vector	В глубину	Беллмана-Форда	3

27	Списки смежности	list	В ширину	Беллмана-Форда	3
28	Матрица смежности	vector	В глубину	Дейкстры	2
29	Списки смежности	list	В ширину	Беллмана-Форда	1

Приложение 2. Примеры для хранения графа

```
std::unordered_map<Vertex, std::vector<Edge>>
std::unordered_map<Vertex, std::list<Edge>>
std::vector<std::vector<Distance>>
std::vector<std::list<Distance>>
```